Foundations of Data Science for Biologists

Dates and times

BIO 724D

2026-NOV-10

Instructors: Greg Wray and Paul Magwene

Challenges in working with dates and times

Many different formats

November 10th, 2026 10 Nov 2026 11-10-25 and many more

3:15pm 15:15 15h 15m and many more

Requires conversion to a standard format, conversion is prone to errors

Non-decimal units, some of which vary in length

60 sec / min 60 min / hr 24 hr / day ? day / month ~365 day / year

Leap years and leap seconds add exceptions in certain situations

Complicates arithmetic with dates and times

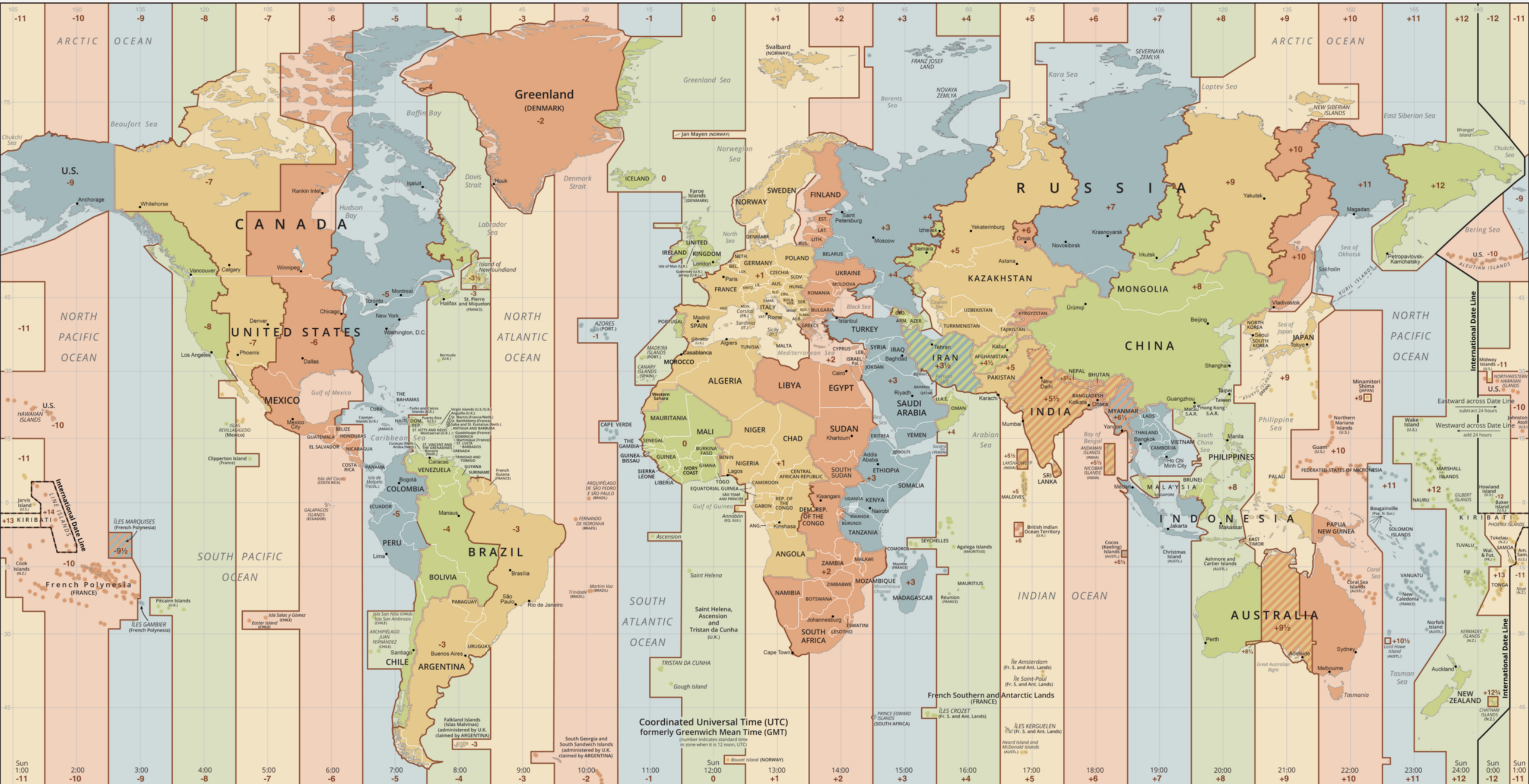
Time zones

No regular organization, many odd exceptions, local standards can change

Daylight savings time introduces further complexity in many locations

Complicates coordination of dates and times, and further complicates arithmetic

Time zones of the world



How do computers store dates and times?

Date and date-time are stored as numeric values

- Number of days or seconds elapsed since `1970-01-01 00:00:00 UTC`
- UTC refers to universal coordinated time, located at longitude `0` (Greenwich, England)

Date and date-time objects also store local time zone as string values

- Local time zone is automatically or manually updated if you change location

Date and date-time objects are represented as strings

- Automatically converts numerics into local date and time for humans
- Automatically adjusts for daylight savings time, leap years, etc.

Computers are not designed to be autonomous clocks

- Eventually, the internal date-time value will drift from true time
- Your OS uses NTP (network time protocol) to synchronize from a time server

ISO 8601: the international standard for dates and times

Defines standards (see: https://en.wikipedia.org/wiki/ISO_8601)

- Units arranged from largest to smallest; unambiguous, allows consistent sorting
- Different separators for dates and times; unambiguous (but optional)
- Consistent number of characters (padded); unambiguous, allows consistent sorting
- Flexible precision: reduce or extend as appropriate

Compliant examples

`"2022-05-14"`	date, extended (standard) format
`"20220514"`	date, basic format
`"2022-05"`	date, reduced precision
`"2022-05-19 15:00:42"`	date-time, extended (standard) format
`"20220519 150042"`	date-time, basic format
`"2022-05-19 15:00"`	date-time, reduced precision
`“2022-05-19 15:00:42:27"`	date-time, increased precision

Outline for notebook on dates and times



Introducing the `lubridate` package

Date and date-time objects

- How to create date and date-time objects
- How to retrieve the current date and date-time for anywhere in the world
- How to convert existing non-ISO 8601 strings into proper date and date-time objects
- How to extract and modify specific components of a date or date-time object

Time spans

- The three time span classes supported by `lubridate` and their use cases
- How to create each class of time span object
- How to modify and carry out mathematical operations with time spans

Date and date-time data types

Value: numeric; units of days or seconds

Representation: string

Format: ISO 8601

Date: days since `1970-01-01`

Representation example: `"2026-11-10"`

Data type: `"Date"`

Date-time: seconds since `1970-01-01 00:00:00 UTC`

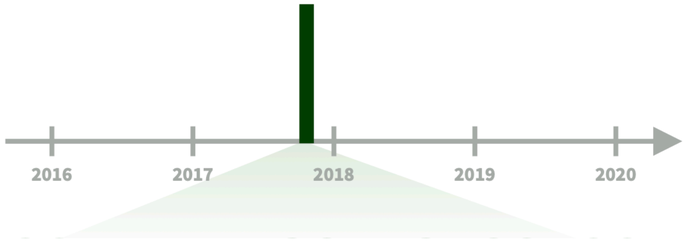
Representation example: `"2026-11-10 15:08:22 EST"`

Data type: `"POSIXct"`

Time

No time-only data type in base R

`lubridate` provides 3 different **time span** data types



`2025-11-11 15:08:15`

date and date-time are unique points along the time-line



`15:08:15`

time alone is not a unique point along the time-line

Time span classes: duration

Definition

A time span defined only by its length

Not anchored to the time-line, an abstract time span

Think of it as: “stopwatch time”



Computationally

Stored as: numeric

Representation: `"79200s (~22 hours)”` , `"5702400s (~9.43 weeks)"`

Ignores the calendar when used in math with date-time objects

Uses

You want to record how long one baboon spends grooming another baboon

You want to compare how long it takes wild type and mutant cells to divide

You want to record time and do math in absolute units and ignore the calendar

Time span classes: period

Definition

A time span defined only by its length

Anchored to the time-line and aware of irregularities

Think of it as: “calendar span”



Computationally

Stored as: numeric (seconds) from a defined start date-time

Representation: `"2M 0S"`, `"5d 4H 12M 47S"`

Consults the calendar when used in math with date-time objects (“intuitive” results)

Period

You are taking field observations every 9 days and want to know the next time point

You are planning a 5-week experiment that requires time points every 30 hours

You want to do date-time math using absolute units, but get results in human units

Time span classes: interval

Definition

A time span defined only by its end points

Anchored to 2 date(-time)s, aware irregularities

Think of it as: “bounded time”



Computationally

Stored as: start date(-time) and end date(-time)

Representation: `"2024-05-01 12:00:00 UTC--2024-05-01 15:32:17 UTC"`

Time difference and other attributes can be extracted with functions

Interval

You have two events recorded in human units and want know the time difference

You want to know if a specific date-time falls within a defined interval

You want to work with time spans defined by their end points rather than duration